Course Outline: Background Processing

Title: Background Processing: Running Work Outside the Request Cycle **Skill:** Skill 45: Background processing **Learnpack:** + Introducción procesamiento en segundo plano (Background processing) **File:** s45-w15d45-introduccion-procesamiento-en-segundo-pl **Prerequisite:** Skill 44 (Building reports from telemetry data) **Target Audience:** Beginner developers building web apps who need to handle tasks that take too long to run inside a request handler **Total Lessons:** 9 **Format:** Conceptual (0% hands-on)

Course Description

Your web app receives a request. The handler needs to send an email, process an image, or generate a report. These take seconds — sometimes minutes. Returning a 200 while they run in the background isn't just a performance trick; it's a fundamental architectural pattern. This course introduces background processing: what it is, when to use it, and the five principles that make it reliable — delegation, idempotency, decoupling, observability, and durability.

Course Philosophy

This course emphasizes:

- Understanding the "why" before the "how" — the principles outlast any specific tool
- Recognizing the problem background processing solves: the long-running task in a short-lived request
- Building a vocabulary for discussing reliability in asynchronous systems

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Beyond the Request Cycle	2
02 - Core Principles	4
03 - Assessment	1
04 - Outro	1
Total	**9**

00.0 Beyond the Request-Response Cycle 🕮

Learning Objectives:

- Identify the structural limitation of the request-response cycle for long-running work

- Recognize what "running in the background" means in a web application context
- Preview the five principles that make background processing reliable

Content Outline:

- The web's fundamental contract: request in, response out, as fast as possible
- The problem: some tasks take too long to run inside a request handler
  - Sending an email confirmation → external service, unpredictable latency
  - Generating a PDF report → CPU-bound, may take seconds
  - Processing an uploaded image → resize, convert, store → multiple steps
- Blocking the request for these tasks means slow responses or timeouts for users
- The solution: delegate the work to a separate process that runs outside the request cycle
- Background processing = executing work that has been decoupled from the triggering request
- Preview: five principles govern whether this decoupling is reliable or fragile — delegation, idempotency, decoupling, observability, durability

Transitions: This lesson frames the problem. Section 01 names it and shows when it applies.

Key Principles:

- The request-response cycle is designed for fast operations; long-running work breaks this contract
- "Running in the background" means the work is not blocking the response — it runs independently
- Decoupling trigger from execution is the core idea behind background processing

Examples:

- User signs up → system returns "Account created" immediately → email confirmation sent 10 seconds later by a background job
- User uploads a video → system returns "Upload received" immediately → video transcoding runs in the background for 2 minutes
- User requests a monthly report → system returns "Report queued" immediately → report generated in the background

---

01.0 What Is Background Processing 🕮

Learning Objectives:

- Define background processing in precise terms: execution outside the request-response cycle
- Distinguish synchronous request handling from asynchronous background execution
- Explain where background jobs fit in a typical web application architecture

Content Outline:

- Precise definition: background processing is the execution of tasks outside the synchronous request-response cycle
- Synchronous flow: request arrives → handler executes → response sent → done

- Asynchronous flow: request arrives → handler delegates task → response sent → task executes separately → done
- The handler's role: not to execute the work, but to hand it off and return
- Where background jobs live: separate process, separate thread, or separate service — not inside the web handler
- The relationship between the trigger (request) and the task (background job): they are linked by a message or queue entry, not by a function call
- Section preview: 01.1 explains when this pattern is needed and when it is overkill

Transitions: This lesson defines the mechanism. The next lesson (01.1) explains when to apply it.

Key Principles:

- Background processing is not about hiding slow work — it's about acknowledging that some work shouldn't block the user
- The response confirms receipt ("queued", "processing"), not completion ("done")
- The trigger and the task may run in different processes, on different machines, at different times

Examples:

- Synchronous: `POST /signup → validate → create user → send email → return 200` (email blocks the response)
- Asynchronous: `POST /signup → validate → create user → enqueue email job → return 200` → (email sent 5 seconds later by background worker)

01.1 When to Use Background Processing 🕮

Learning Objectives:

- Apply decision criteria to determine when a task should be moved to the background
- Identify common use cases that always benefit from background processing
- Recognize tasks that do NOT benefit from background processing

Content Outline:

- The key question: "Can the user wait for this to finish before getting a response?"
  - If yes → synchronous is fine
  - If no → background processing is needed
- Use background processing when:
  - The task involves external services with unpredictable latency (email providers, SMS services, third-party APIs)
  - The task is computationally expensive (image processing, PDF generation, data export)
  - The task affects resources outside the request's scope (updating multiple database tables, sending notifications)
  - The task needs to be retried on failure (you can't retry a synchronous handler without repeating the request)
  - The task is scheduled for a later time (not triggered by a user action at all)
- Don't background-process when:

    - The result must be returned to the user in the same response (validation, authentication, data fetching)
    - The task is fast enough to not matter (under 100ms)
    - The complexity of asynchronous handling would outweigh the benefit

Transitions: You now know when to use it. Section 02 teaches the five principles that make it reliable.

Key Principles:

- Background processing adds complexity — only use it when the alternative (blocking the response) is worse
- External service calls are the most common trigger for moving work to the background
- If the task must retry on failure, background processing is the right architectural choice

Examples:

- Use background: email confirmation, image resizing, PDF report generation, webhook delivery, scheduled data pipeline
- Don't use background: form validation, fetching user data for a dashboard, returning search results

---

02.0 Core Principles 📖

Learning Objectives:

- Name the five core principles of reliable background processing
- Explain what each principle addresses and why it matters
- Understand how these principles work together to prevent common failure modes

Content Outline:

- Background processing fails in predictable ways: tasks run twice, tasks fail silently, tasks get stuck, nobody knows what happened
- The five principles are the answers to these failure modes:
    1. **Delegation** — move the task out of the request handler
    2. **Idempotency** — design tasks that are safe to run multiple times
    3. **Decoupling** — producer and consumer operate independently
    4. **Observability** — know what happened and when
    5. **Durability** — tasks complete despite failures in the system
- Section preview: 02.1 covers delegation and decoupling; 02.2 covers idempotency; 02.3 covers observability and durability
- Why all five are needed: missing any one creates a class of failures the others can't prevent

Transitions: This overview sets up the next three lessons, each of which deep-dives into a subset of principles.

Key Principles:

- These five principles are tool-independent — they apply whether you use a queue, a cron job, a worker, or a message broker
- Each principle addresses a specific failure mode; understanding the failure mode clarifies why the principle matters
- Together they define what "reliable background processing" means

Examples:

- Missing delegation: slow response (task blocks request handler)
- Missing idempotency: duplicate records when a task retries (email sent twice, invoice charged twice)
- Missing decoupling: consumer crash brings down the producer
- Missing observability: task fails silently, no one knows
- Missing durability: task stops mid-execution and leaves data in a broken intermediate state

---

02.1 Delegation and Decoupling 🕮

Learning Objectives:

- Define task delegation as moving work out of the request handler and into a separate executor
- Define decoupling as the independence of producer and consumer
- Explain why coupling the producer to the consumer creates fragility

Content Outline:

- **Delegation — the "move" principle:**
  - The request handler's only job is to receive the request, validate it, and hand off the work
  - The handler does NOT execute long-running tasks — it creates a job description and passes it to a worker
  - Delegation is the action; background processing is the result
  - What gets delegated: the full task specification (inputs, context, what needs to happen)
  - What the handler keeps: creating the job record, returning a response to the user
- **Decoupling — the "independence" principle:**
  - Producer (the part that creates jobs) and consumer (the worker that runs them) are independent
  - Decoupling means: if the consumer crashes, the producer keeps working; if the producer is slow, the consumer processes at its own pace
  - Coupling is the anti-pattern: if worker goes down, requests start failing; if the request spike overwhelms the worker, jobs pile up
  - Message queues and job queues achieve decoupling by acting as the intermediary
  - "Colas de mensajes" (message queues) from the curriculum: the queue is the decoupling mechanism
  - Note: queues are covered in depth in Skill 46 — here we understand why decoupling matters, not how queues work internally

Transitions: Delegation and decoupling are structural principles — they define the architecture. The next lesson (02.2) addresses what happens when tasks run more than once.

Key Principles:

- The handler produces work; the worker consumes work — they should be independent
- A decoupled system is resilient: one component can fail without bringing down the others
- Delegation is the act of handing off; decoupling is the guarantee that the hand-off doesn't create a dependency

Examples:

- Coupled (bad): `send_confirmation_email(user)` called directly from the request handler — if the email service is down, the request fails
- Decoupled (good): `job_queue.enqueue({"task": "send_email", "user_id": user.id})` — if the email service is down, the job waits; the request succeeds
- Real-world analogy: a restaurant places orders on a kitchen ticket — the waiter (producer) is decoupled from the chef (consumer); if the kitchen is slow, the waiter keeps taking orders

---

02.2 Idempotency 🕮

Learning Objectives:

- Define idempotency in the context of background tasks
- Explain why background tasks are almost always retried and why idempotency is therefore essential
- Design a simple idempotent task using an idempotency key

Content Outline:

- **Why tasks retry:** network failures, worker crashes, timeouts — these are not exceptional; they are expected in distributed systems
- A retry means the same task runs more than once — the question is whether running it twice produces the same result as running it once
- **Idempotency definition:** a task is idempotent if running it multiple times produces the same outcome as running it once
- The key insight: the problem is not that retries happen — it is that non-idempotent tasks produce wrong results when retried
  - Non-idempotent: `INSERT INTO invoices VALUES (...)` on retry → duplicate invoice
  - Idempotent: `INSERT INTO invoices VALUES (...) ON CONFLICT DO NOTHING` on retry → safe
- How to design idempotent tasks:
  1. Check before acting: "Has this task already run for this input?"
  2. Use idempotency keys: a unique identifier per task instance (e.g., user_id + event_type + date) that prevents duplicate processing
  3. Use upserts instead of inserts: `INSERT ... ON CONFLICT DO UPDATE` or `UPDATE ... WHERE NOT EXISTS`
- **Idempotency ≠ deduplication:** idempotency is about the outcome; deduplication is about preventing the retry entirely — both are useful, idempotency is more robust

- Avoiding data duplication (from the curriculum bullet "Evitar la duplicación de datos") is the direct consequence of designing idempotent tasks

Transitions: Idempotency protects against corrupted outcomes when tasks run more than once. The next lesson (02.3) addresses knowing what happened (observability) and guaranteeing completion (durability).

Key Principles:

- In distributed systems, "at least once" delivery is the norm — assume every task will be retried at some point
- Idempotency is not a nice-to-have; it is a correctness requirement for any task that touches state
- An idempotency key is the simplest mechanism: if the key has been processed, skip; otherwise process and record the key

Examples:

- Non-idempotent: charge a credit card (running twice charges twice)
- Made idempotent with a key: check if `payment_id` already exists in `payments` before charging — if it does, return the existing result
- Non-idempotent: send a welcome email to user_id 42 (running twice sends two emails)
- Made idempotent: check if `welcome_email_sent` flag is already set for user_id 42 before sending

---

02.3 Observability and Durability 🕮

Learning Objectives:

- Define observability as the ability to know what a background job did, when it ran, and whether it succeeded
- Define durability as the guarantee that a task runs to completion despite failures
- Explain retry logic and error handling as the practical implementation of durability

Content Outline:

- **Observability — "poder saber qué pasó" (the ability to know what happened):**
  - Background jobs are invisible by default — they run in a different process, often without direct user visibility
  - Observability means: at any point, you can answer "Did this task run? When? Did it succeed or fail?"
  - Minimum observability requirements per job: timestamp of execution, status (success/failure), error message if failed
  - Structured logging: every job execution should produce a log entry with these fields
  - Job status tracking: a `status` column in a jobs table (pending → running → completed/failed) is a simple observability pattern
  - Why it matters: without observability, failures are silent — you discover them when users complain, not when they happen
- **Durability — "que la función no se detenga en su ejecución":**

    - Durability means: if a background job is interrupted mid-execution (server crash, network outage), it will eventually complete
    - The mechanism for durability: persistence + retry
    - Persistence: the job must be stored (in a database or queue) before it starts — if the worker crashes, the job can be re-picked by another worker
    - **Retry Logic:** automatic re-execution on failure; backoff strategies prevent hammering a failing dependency
        - Fixed retry: try again after N seconds
        - Exponential backoff: wait longer between each retry (1s → 2s → 4s → 8s...)
        - Maximum retries: after N attempts, mark the job as dead (moved to a "dead letter" holding area for manual review)
    - **Error Handling:** errors should be caught, logged, and used to determine retry eligibility — not silently swallowed
    - **Persistence (as a best practice):** save intermediate results when processing large data in chunks — if interrupted, resume from the last saved checkpoint rather than starting over
    - Best practices woven in: always log before and after job execution; treat every background job as if it might be interrupted at any point; design the system to recover from any interruption without human intervention

Transitions: All five principles together define what it means for background processing to be reliable. The assessment tests your ability to apply them to new scenarios.

Key Principles:

- Observability is the feedback loop for background processing — without it, failures accumulate silently
- Durability requires both persistence (jobs survive restarts) and retry logic (jobs eventually complete)
- Retry logic + error handling + logging are not optional extras — they are the minimum viable implementation of durability

Examples:

- Observability log entry: `{job_id: "j-abc123", task: "send_email", user_id: 42, started_at: "2024-01-15T14:23:00Z", status: "failed", error: "SMTP timeout"}`
- Retry with backoff: attempt 1 at T+0s, attempt 2 at T+2s, attempt 3 at T+4s, attempt 4 marked as dead
- Durability via persistence: before processing a 10,000-row CSV, save progress every 100 rows — if the job crashes at row 350, resume from row 300

03.0 Assessment 🤔

Learning Objectives:

- Identify which principle each design decision implements
- Apply the principles to diagnose failure modes in scenario descriptions
- Choose the correct principle-based solution for a given problem

Question Format: Scenario-based (multiple-choice)

Questions:

1. A payment processing job runs, charges the user, and then crashes before marking the payment as complete. On retry, the job charges the user again. Which principle was violated?

    - A) Delegation
    - B) Observability
    - C) Idempotency ✓
    - D) Decoupling

2. An email notification worker crashes, and immediately all new user signups start failing because the signup handler cannot continue without the worker. Which principle is missing?

    - A) Durability
    - B) Decoupling ✓
    - C) Idempotency
    - D) Observability

3. A background job generates a weekly report. It runs every Monday but nobody knows if it succeeded or failed until a user notices the report is missing. Which principle is missing?

    - A) Delegation
    - B) Durability
    - C) Idempotency
    - D) Observability ✓

4. A video transcoding job takes 30 seconds. The developer runs it directly inside the POST /upload request handler. Users experience 30-second timeouts on upload. Which principle would fix this?

    - A) Delegation ✓
    - B) Idempotency
    - C) Observability
    - D) Durability

5. A background job processes telemetry events but crashes midway through. When restarted, it re-processes all events from the beginning, causing duplicate entries. What is the most appropriate fix?

    - A) Add retry logic with exponential backoff
    - B) Save a progress checkpoint after each batch and resume from the last checkpoint ✓
    - C) Increase the worker's memory allocation
    - D) Remove the retry logic to prevent duplicate processing

6. Which of the following is NOT a correct way to make a task idempotent?

    - A) Check if a record with the same idempotency key already exists before creating a new one
    - B) Use `INSERT ... ON CONFLICT DO NOTHING` for database inserts

    - C) Run the task exactly once by disabling retries ✓
    - D) Mark the task as "processed" after completion and skip it if the mark exists

7. A background job sends an email but has no logging or status tracking. The job fails silently. What is missing?

    - A) Decoupling
    - B) Durability
    - C) Observability ✓
    - D) Delegation

8. A background job fails due to a temporary network outage. The correct durability pattern for re-running the job is:

    - A) Alert the developer to run the job manually
    - B) Delete the job and ask the user to retry their action
    - C) Automatically retry the job with exponential backoff ✓
    - D) Increase the request timeout and run it synchronously next time

Evaluation Criteria:

- Correctly maps each scenario to the relevant principle
- Understands idempotency as a correctness requirement, not an optimization
- Recognizes observability as the diagnostic tool and durability as the recovery mechanism

Score Interpretation:

- 8/8: You can design reliable background processing systems
- 6–7: Review any missed principles before moving on
- 4–5: Re-read Section 02 with focus on the principle → failure mode mapping
- Below 4: Restart from Section 01

04.0 Reliable Background Jobs

Content Outline:

- Course recap: what students accomplished
    - Named the structural problem: long-running tasks block the request-response cycle
    - Understood when background processing is the right architectural choice
    - Learned all five principles: delegation, idempotency, decoupling, observability, durability
    - Connected each principle to the failure mode it prevents
- Connection to bigger picture: these five principles are tool-independent — they apply to cronjobs, message queues, worker pools, serverless functions, and every other background processing mechanism
- Next steps: Skill 45 continues with trigger types (how background jobs get started) and cronjob-based scheduling — the next two learnpacks apply these principles to concrete implementations
- Resources for further exploration:
    - "Building Microservices" by Sam Newman — chapters on asynchronous communication

    - AWS architecture docs on async patterns — practical examples of these principles in production
    - Any background job library documentation (Celery, Sidekiq, BullMQ) — all organize their features around these same five principles
- Encouragement: the five principles feel abstract now. In the next learnpacks you'll see exactly how cronjobs and queues implement each one — and recognizing the principle in the implementation is the skill that transfers to every new tool you encounter.

Note: This is conclusion only — no theory, exercises, or assessment.